Load Balancer Fundamentals

one address in front, many servers behind

Watch the 4-min video

In one line: a load balancer gives clients one address, spreads their requests over a pool of healthy servers, and steers around failed ones; it often terminates TLS too, and it must be redundant itself, or it becomes the single point of failure.

1 Why you need one

One server can't keep up anymore, so you add three more. Two problems appear at once. Clients need to know **which server** to call, and when one of them crashes, someone must **stop sending it traffic**. Hard-coding four addresses in every app doesn't solve either.

A **load balancer** (LB) solves both. It sits between the clients and a **pool** of servers (also called *backends* or *targets*). Clients connect to one address: a **virtual IP** (VIP, an address not tied to one machine) or a DNS name. For each connection or request, the load balancer picks a healthy backend and forwards the traffic to it.

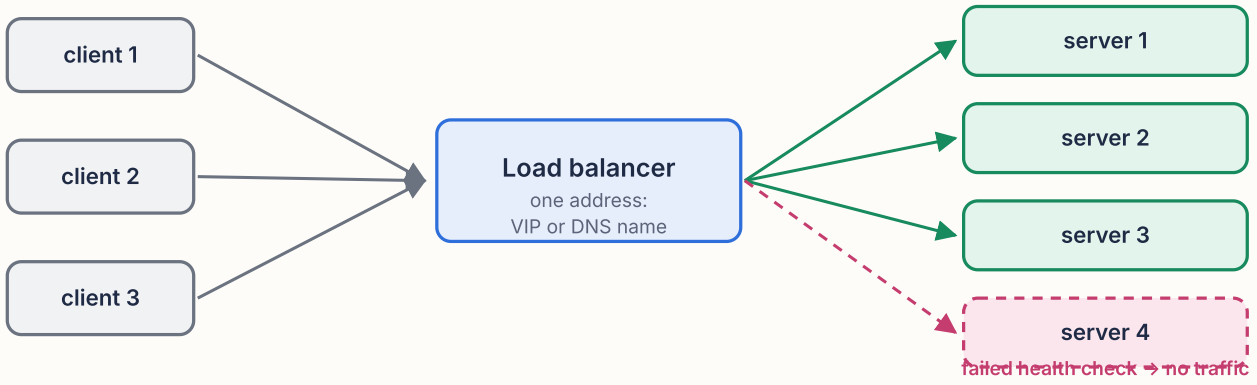


Figure 1. Clients see one address. The load balancer forwards each request to a healthy server; a server that fails its health check gets no traffic.

Because clients only know the load balancer's address, you can add, remove or replace servers behind it and no client notices. That's what makes horizontal scaling (Phase 1, lesson 7) practical.

2 More than spreading load

Most load balancers do several other jobs, which is why nearly every production system has one, and often several:

- **Health checks.** It probes each backend and removes failed ones automatically (lesson 7 of this phase).
- **TLS termination.** It holds the certificate and decrypts HTTPS, so backends don't have to.
- **Connection draining.** During a deploy, it stops sending *new* requests to an old server but lets in-flight ones finish.
- **Timeouts and limits.** One place to cap request time, body size and request rate.
- **Observability.** One place for access logs and metrics: request rate, errors, latency per backend.



one place in front of the servers, so every server doesn't have to do these jobs itself

Figure 2. The five everyday jobs of a load balancer. Doing them once, in front, keeps every backend simpler.

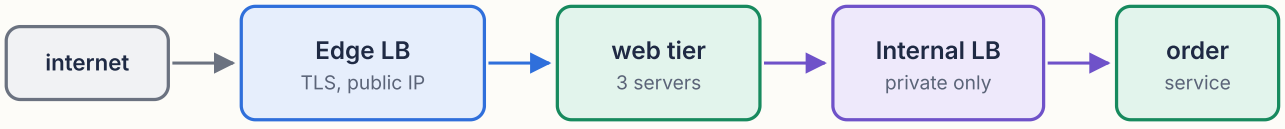
3 Three kinds of load balancer

Kind	Examples	Good	Watch out
Hardware appliance	F5 BIG-IP, Citrix ADC	very fast, dedicated chips	expensive, slow to change, you buy capacity up front
Software	NGINX, HAProxy, Envoy	runs on ordinary servers, flexible, open source	you run, patch and scale it
Cloud, managed	AWS ALB and NLB, Google Cloud Load Balancing, Azure Load Balancer	no servers to manage, scales automatically, multi-zone	provider limits and pricing per hour and per traffic unit

Most new systems use a cloud load balancer at the edge and software proxies (often Envoy) inside.

4 Where it sits: edge and internal

An **edge** load balancer faces the internet: it has the public IP, terminates TLS, and takes traffic from browsers and apps. **Internal** load balancers sit between your own tiers, like the web servers calling an order service; they have private addresses only. In microservices, a **service mesh** (Istio, Linkerd) often does internal balancing in a *sidecar* proxy that runs next to every service.



in microservices, a service mesh sidecar next to each service often does the internal balancing

Figure 3. An edge load balancer at the internet boundary and an internal one between tiers. Large systems have many of each.

5 Isn't the load balancer a single point of failure?

Yes, if there is only one. All traffic goes through it, so it must be redundant:

- **Active-passive pair with a floating VIP.** Two load balancers share one virtual IP. The active one holds it; the standby listens for its heartbeats with a protocol like **VRRP** (Virtual Router Redundancy Protocol, used by keepalived). If the heartbeats stop, the standby claims the IP within seconds.
- **Several active nodes.** Many load balancer nodes all take traffic, spread by DNS (several A records) or by **anycast** (the same IP announced from many places).
- **Managed cloud load balancer.** AWS, Google and Azure run it on many machines across availability zones for you.

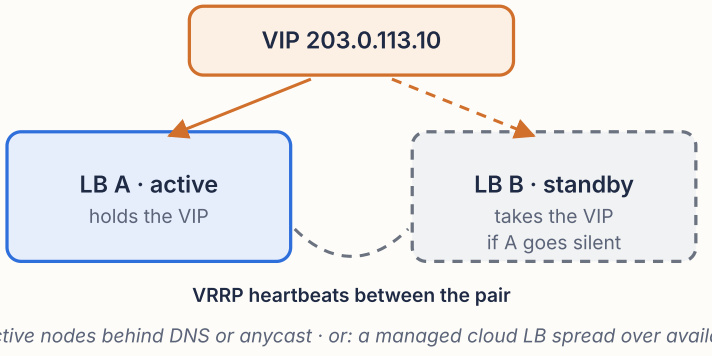


Figure 4. An active-passive pair: the active node holds the virtual IP; if its VRRP heartbeats stop, the standby takes the IP over.

WORKED EXAMPLE: AN ALB IN FRONT OF ECS

An AWS Application Load Balancer is spread across three availability zones, in front of an ECS service (containers). Each new task registers itself in a **target group** automatically. The ALB health-checks every task and sends requests only to healthy ones. During a deploy, it **drains** the old tasks (no new requests, in-flight ones finish, up to a deregistration delay such as 30 seconds) while new tasks start. The ALB itself is managed and redundant across the zones, so it's not your single point of failure.

COMMON MISTAKES

- Drawing one load balancer box in an interview and never saying how it is made redundant.
- No health checks, or checks that only test that the process is up, so broken servers keep getting traffic.
- Deploying without draining, which cuts off requests that were in flight.
- Keeping user sessions in server memory, so the load balancer can't move a user to another server (lesson 6 covers sticky sessions).

6 Key terms

Load balancer: spreads requests over a pool of servers behind one address

Virtual IP (VIP): an address that can move between machines

TLS termination: the LB decrypts HTTPS so backends get plain HTTP

VRRP: protocol that moves a VIP to the standby when the active fails

Target group: the AWS name for a pool of backends

Backend / target: one server in the pool

Health check: a probe that marks a backend healthy or not

Connection draining: finish in-flight requests before removing a server

Anycast: one IP announced from many locations

7 Interview questions

Q1. What does a load balancer do? BASIC

It gives clients one address and spreads their requests across a pool of healthy servers, removing failed ones with health checks. Most also terminate TLS, drain connections during deploys, enforce timeouts and limits, and give one place for logs and metrics.

Q2. How do you make the load balancer itself highly available? BASIC

Run it redundantly: an active-passive pair sharing a floating virtual IP with VRRP, several active nodes behind DNS or anycast, or a managed cloud load balancer that runs across availability zones.

Q3. Hardware, software or cloud load balancer: how do you choose? BASIC

Hardware is fast but expensive and slow to change. Software (NGINX, HAProxy, Envoy) is flexible but you run it. Cloud load balancers are managed and scale automatically, so they are the usual default for new systems, with software proxies inside.

Q4. You deploy a new version and some users see errors for a few seconds. Why, and how do you fix it? SCENARIO

Old servers were removed while requests were still in flight, or new servers got traffic before they were ready. Enable connection draining with a deregistration delay, and use a readiness health check so a new server only gets traffic once it can serve.

Q5. Where do you put load balancers in a three-tier web app? SCENARIO

An edge load balancer at the internet boundary in front of the web tier, terminating TLS, and an internal load balancer (or a service mesh) in front of each internal service the web tier calls. Each is redundant across zones.

Q6. A client keeps one long-lived connection. Does the load balancer still spread its requests? TRICKY

Not at layer 4: the connection is pinned to one backend when it opens. A layer 7 load balancer can spread individual HTTP/2 or gRPC requests from that one connection. That's the next lesson.

Q7. The load balancer is up, but every request times out. What do you check? TRICKY

The health checks (is any backend marked healthy?), the target group's security groups and ports, and the load balancer's idle and request timeouts versus how long the backend takes. A healthy LB with no healthy targets returns errors such as 502 or 503.

Next: SD-G-002 · Layer 4 vs Layer 7 Load Balancing backendroadmap.com